

5-CLASS SERIES · 專為香港設計 · OLLAMA + CLAUDE CODE

# Your Own AI Agent

自己整個私人 AI 助理 · 香港用到 · 第 1 堂

私人 AI Agent ( 香港版 ) · Class 1 — 概念 + Agent + Workspace + Telegram

01 / 32

# 今堂你會擁有

---

## 一堂搞掂：有腦、有屋企（workspace）、有手機收發

- 識乜嘢係 personal AI agent、佢點運作
- 一個喺自己部機跑、用 **kimi-k3:cloud**（**Ollama cloud**）嘅 working agent
- 起好你嘅 **workspace**：work / family / personal, 各有 CLAUDE.md, 共用 SOUL.md / MEMORY.md
- 駁好手機： Telegram —— 手機直接同你個 agent 對話
- 完全唔使 **ChatGPT / Claude** 戶口（香港都用到）

# 五堂路線圖 · 逐堂砌起你個 agent

---

成個系列點行,一眼睇晒 —— 今堂打底,之後逐堂加能力

- **Class 1** ( 今堂 ) 個腦 + 個殼 + workspace +  Telegram —— 一個 working agent
- **Class 2** Skills & Tools —— 可重用技能 + 駁第一個外部工具 Gmail ( MCP )
- **Class 3** 記憶 + Agentic workflow —— 叫佢自己升級自己,整一套記得你嘅記憶
- **Class 4** 語音 · 視覺 · API —— 聽 / 睇 / 講 + 加雲端能力落你個 project
- **Class 5** 自主 + Guardrails —— Loops 自動化 + 規則紅線管得住佢

# 01 Agent 係點組成

先搞清楚三件嘢——之後五堂,堂堂都用得着

今堂最緊要嘅句

# 模型自己乜都執行唔到

佢由頭到尾只係吐文字,講「我想做呢件事」。真正出手嘅,係外面嗰個程式。呢句去到第五堂放佢自主嗰陣,仲係喎。

# Agent = 模型 + 迴圈 + 工具

## 模型

個腦 · 負責諗

- 讀你俾佢嘅嘢
- 決定下一步做咩
- 只係吐字,唔會郁

## 迴圈

個引擎 · 負責再嚟一次

- 做完一步睇返結果
- 錯咗有得改
- 冇天然終點,要你封頂

## 工具

對手 · 負責做

- 讀檔、發訊息、開網頁
- 由你個程式執行
- 俾幾多就做到幾多

**模型自己乜都執行唔到。**

佢由頭到尾只係吐文字,講「我想做呢件事」—— 真正出手嘅,係外面嗰個程式。

# 點解一定要有個迴圈？

---

## 一次過答 vs 做完再睇

- **一次過答 = 盲估** —— 模型要喺乜都未見過嘅情況下,寫晒成個計劃出嚟
- **有迴圈,佢做錯之後見到自己做錯** —— 第三步報錯,第四步先有得改。呢個分別唔係聰明啲,係有咗回饋
- **改錯竅門:將 **error** 原文原封不動貼返俾佢**,唔好改寫成「失敗咗」—— 佢讀 stderr 改錯,叻過讀你嘅摘要
- **三個死法:唔收斂**(同一步試完又試)、**context 爆**(每步都貼落對話,永遠唔會縮)、**錯上加錯**
- **迴圈冇天然終點** —— 佢淨係「唔再叫工具」就停。所以一定要封頂:最多幾多步、幾多錢、幾耐 timeout

# Chatbot vs Agent —— 差喺邊？

## Chatbot

- › 你問先答,答完就停
- › 冇記憶,下次由零開始
- › 淨係傾偈 —— 佢話你知點做
- › 做極都係一段文字

## Agent

- › 俾個目標佢,佢自己行到完
- › 有記憶,而且係你 own 嗰份
- › 識用工具 —— 佢真係去做
- › 識自己 cron,唔使你叫

# 你個 agent 由四層砌成

## 你公司 / 客戶嘅標準

格式、審批、交付規矩

## Domain knowledge

規範、術語、你點判斷

## 你行業嘅 workflow

真實流程,唔係示範

## 基本 agent

模型 + 迴圈 + 工具 —— 今堂砌起

## 你嘅價值

同一個 base agent,  
落到你手先變成  
做得到你份工嗰個。

所以五堂嘅內容,  
係跟你嘅 use case 編。

# 02 Demo · 我自己用緊嘅五樣

同一個 agent, 唔同嘅工 —— 分別喺你教咗佢啲乜

# Demo 1 · 一句話 → 一份 PCI 草稿

Agent 讀到嘅嘢(你個 project folder)

Programme

平面圖



既有報告

往來 email

你講一句: 「幫我寫呢個 project 嘅 PCI」



PCI

(草稿)

跟公司模板:章節、編號、用語一致  
只揀出屬於 PCI 嘅資料,唔關事嘅唔會塞入去

**04 · 判斷同簽名嗰步仍然係你**  
佢出草稿,唔係出決定。

(上面幾個 thumbnail 係示意 —— 真文件喺工作 workspace,冇擺出嚟。)

## Demo 2 · SketchUp 出 view → 專業效果圖

SketchUp 出嚟嘅 view



一句話之後, Gemini 出嘅圖



**幾何、招牌、材料 —— 一樣都冇變, 只係變真咗**

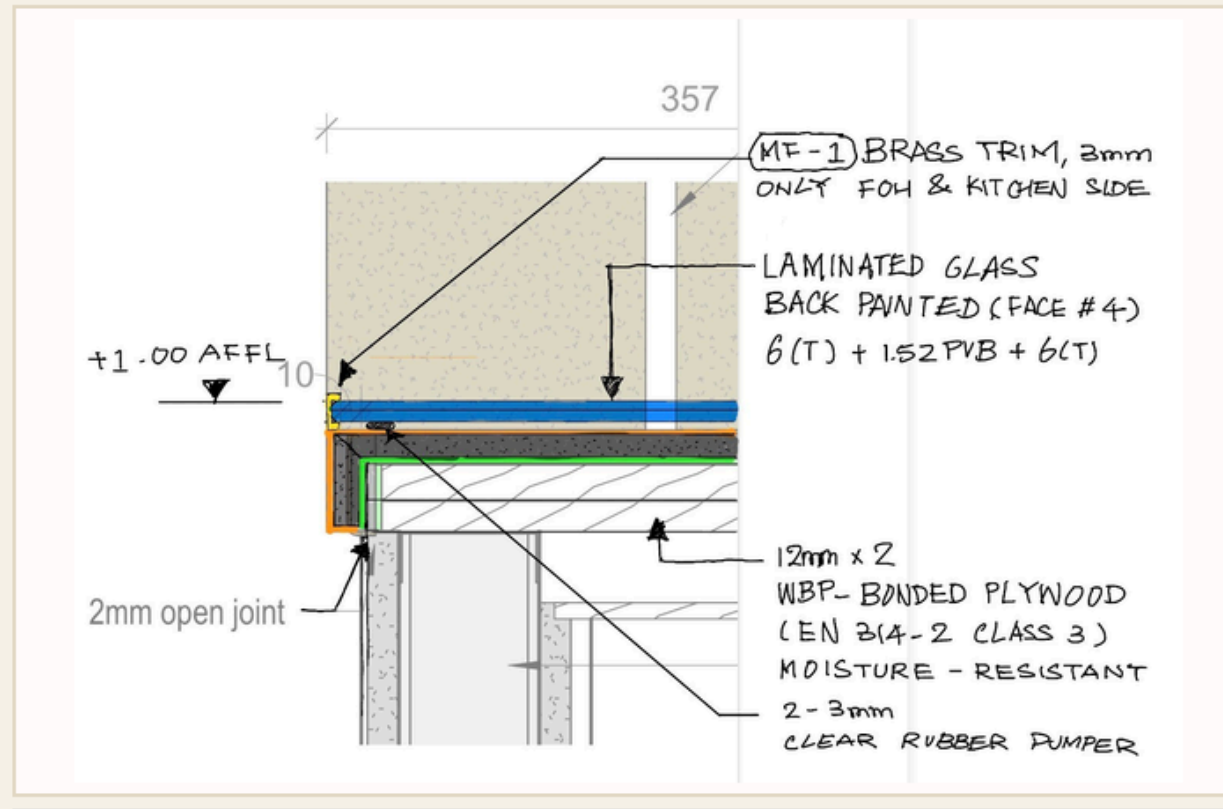
呢個唔係「畫返張新圖」。你要明確要求佢保留 geometry、graphics、材料同鏡頭, 否則佢會自由發揮。

**一套圖要一個 master + references**

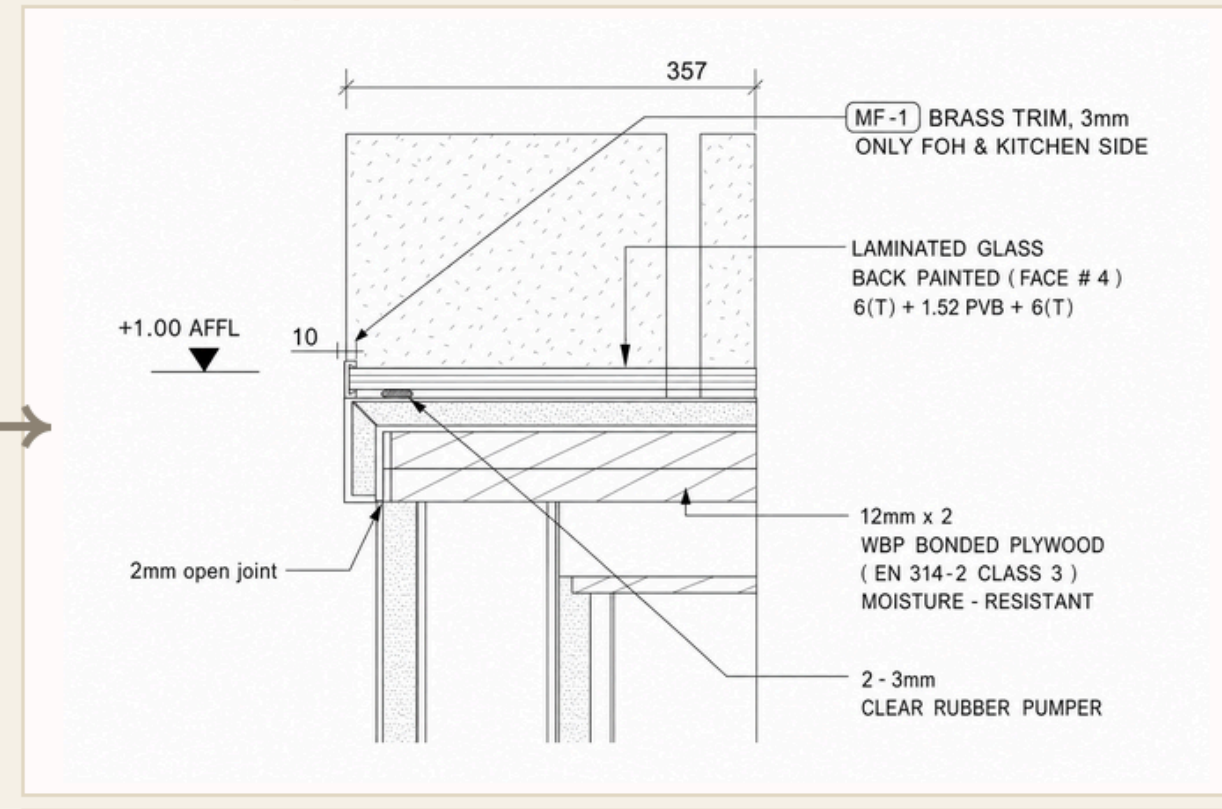
逐張獨立生, 同一間舖每張都唔同款。呢啲係你行業嘅 workflow —— model 唔會自己知。

# Demo 3 · 手畫草圖 → 專業 detail drawing

我真係手畫嗰張草圖



同一張草圖, AI 重畫成 detail



每一個標註都原封不動搬咗過去

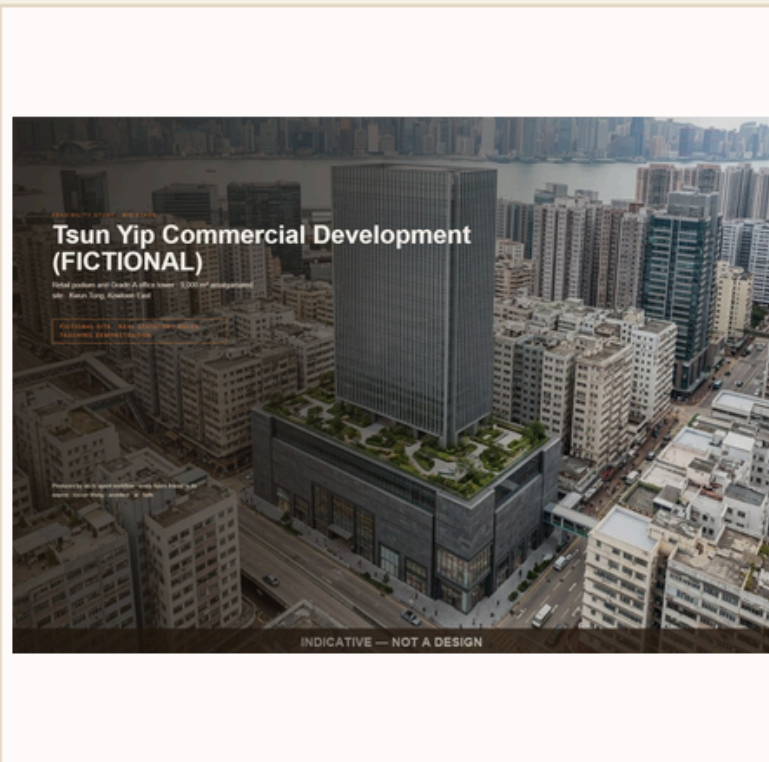
MF-1 黃銅收邊、6(T)+1.52 PVB+6(T) 夾膠玻璃、EN 314-2 Class 3 夾板、2mm open joint —— 一個字冇改。

但要記住:呢張係一幅「圖片」,唔係 CAD 檔

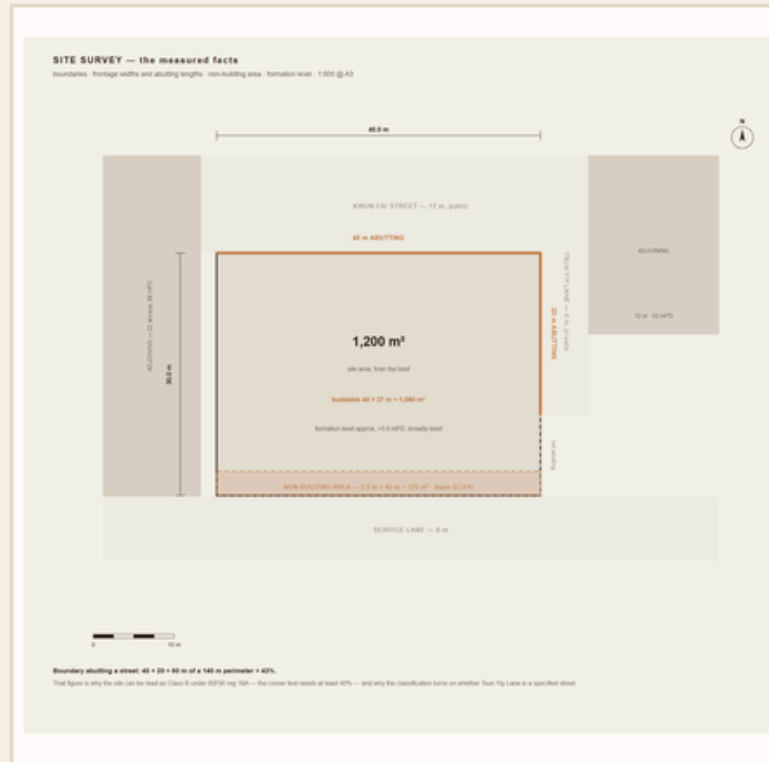
線同數字係生成出嚟嘅,好似樣但唔可以當真。落得手嗰張,幾何要由程式(ezdx)照住你俾嘅數畫。

# Demo 4 · 一個指令 → 一份可行性研究報告

## 封面



## Site survey 圖



報告内頁



每一個數都有出處

報告入面每個數字都追返去佢嘅來源文件——唔係 AI 憑感覺寫，係讀完資料再計。

而佢係喺一個虛構地盤上面,行真嘅法例

教學用途要咁做:條例係真、地盤係假,先可以攤出嚟講。

# Demo 5 · 現場相 → 佢睇得明,亦都有睇唔到嘅嘢

我自己影嘅現場相



佢睇完之後講返俾你聽

- 現況  
轉角舖 · 現有食肆 · 凹入式入口,舖身窄而深
- 立面  
深色釉面磚基座 · 拱形窗楣 · 突出式招牌箱
- 街道  
行人專用區 · 石板鋪面 —— 送貨時間會受限
- 無障礙  
門檻近乎平口,入口冇級
- 鄰接  
左:運動服零售 · 右:珠寶零售 —— 人流性質唔同
- 要跟進  
招牌 / 雨篷要唔要 consent · 樓上用途 · 排煙位

呢啲全部係佢由張相讀返出嚟嘅,唔係我打入去。

但要記住條界:

相入面睇唔到嘅嘢 —— 結構、樓面淨高、機電、樓上實際用途 —— 佢一律唔知,亦都唔會話你知佢唔知。

佢做到嘅係「幫你睇一次、列晒出嚟」,唔係代替現場勘測。

# 五個 demo 嘅共通點:佢知你個 project

## 個 model 叻 ≠ 幫到你

- › Chatbot 知全世界,但唔知你個 project
- › 所以佢寫得出「一般嚟講」,寫唔出「呢個 project」
- › 你要嘅唔係一個識答嘢嘅人,係一個跟開你單嘢嘅人
- › 呢樣叫 **context** —— 佢對呢件事嘅認識

## 點樣入到佢度

- › 放埋喺同一個 **folder** —— 佢有工具,自己會去讀
- › 寫低規矩(**CLAUDE.md**) —— 呢個 project 係咩、要跟咩格式
- › 決定要寫落檔 —— 只講過一次嘅嘢,下次對話就冇咗
- › 唔使一次過餵晒 —— 每次佢自己揀相關啲幾份嚟睇

呢個先係分別

# 佢知道呢個 **project** 要有啲咩設施

所以你叫佢排 layout 嘅時候,佢會自己擺返啲設施入去,唔使你每次提。同一個 model,知你個 project 個個,同唔知嘅個個,係兩件事。

# 03 落手砌

由今堂開始,底層嗰個 agent 係你自己嘅

# 實作 · 裝 Ollama(個腦)

準備:personal laptop(唔好用公司機)+ 穩定 WiFi + ollama.com 戶口。呢部分係裝軟件,要打指令;裝完之後基本上叫 agent 做就得。

## ① 裝 OLLAMA 並登入

# 裝 Ollama — 揀返你部機系統:

# Mac: ollama.com/download 下載 app (或 brew install ollama)

# Windows: ollama.com/download 下載 OllamaSetup.exe (或 winget install Ollama.Ollama)

# Linux: curl -fsSL https://ollama.com/install.sh | sh

ollama --version # 裝好後確認

ollama signin # cloud model 要先喺 ollama.com 開戶口

ollama signin 要先去 ollama.com 開戶口(cloud model 用)。

# 實作 · 裝 Claude Code(個殼)

claude.ai 個一鍵 installer 喺香港被封,所以改用唔被封嘅裝法 —— Windows 用 winget,Mac 用 Node(nodejs.org)+ npm:

裝 CLAUDE CODE · 揀返你部機

# Windows:

```
winget install Anthropic.ClaudeCode
```

# Mac:(先去 nodejs.org 下載 LTS .pkg 裝 Node)

```
npm install -g @anthropic-ai/claude-code
```

```
claude --version      # 確認(重開 terminal 刷 PATH)
```

claude.ai 個一鍵 installer 喺香港被封,所以 Windows 用 winget、Mac 用 Node(nodejs.org)+ npm。CLI 本身喺香港 run 得(連 Ollama,唔連 Anthropic)。

# 實作 · 接個腦上線 + 第一次對話

一條指令接好個腦 ( kimi-k3:cloud —— 識用工具、識睇圖 ), 然後第一次用英文 **prompt** 同佢講嘢：

## ① 接 OLLAMA 個腦

```
ollama launch claude --model kimi-k3:cloud  
# kimi-k3:cloud — 又快又叻、識睇圖、識用工具; 唔使勁機; **要先 ollama signin**
```

## ② 喺 CLAUDE CODE 入面打 ( 英文 PROMPT )

```
hello, what model are you?  
make a folder ~/my-agent and write hello.txt in it  
saying "I built my own AI agent today"
```

cloud model 跑喺 Ollama 部 GPU, 部機唔使勁; 但一定要 `ollama signin` 登入, 否則會靜靜雞 hang。你冇自己開 **folder/寫 file** —— 用英文叫佢, 佢落手。

# 揀邊個 model ? cloud 定本地

起步用 kimi-k3:cloud(1M context、識睇圖、tool calling);想 \$0/私隱就行本地;要重型寫 code 就上 ollama.com 揀現行 coding model

-  起步推薦 · **cloud** → kimi-k3:cloud(**1M context**、識睇圖、tool calling)—— demo 預設;要先 **ollama signin**
-  **注意**:ollama.com 標住 kimi-k3 係 **Extra High Usage** —— 用量燒得快過細 model,唔好開住佢通頂跑
-  **重型寫 code** → 上 ollama.com 揀個現行 coding model(model 名會變,唔好背死)
-  **想 \$0 / 私隱 / 唔使登入** → 本地 model(例如 gemma4:31b,~16-24GB RAM);要夾返你部機 RAM
-  cloud 一定要 `ollama signin`,唔係會 hang(冇返應);切換好易,改 `--model` flag 就得

# 工具 · Telegram ①(攞 token + chat id, 裝 plugin)

你親手只需做呢幾步 —— 攞兩個資料 + 一次過裝 plugin。Bun 同其餘 setup 等陣全部交俾 agent。

## ① 親手攞兩樣嘢

# Token — Telegram 搜 @BotFather → /newbot → 改名

# → 攞個 token, 似 123456789:AAHfiqksKZ8...

# Chat id — Telegram 搜 @userinfobot → DM 佢

# → 佢即刻覆你個 numeric id, 似 412587349

## ② 一次過裝 PLUGIN(CLI, 打一次)

```
/plugin marketplace add anthropics/claude-plugins-official
```

```
/plugin install telegram@claude-plugins-official
```

```
/reload-plugins
```

兩個資料(token + chat id)抄低, 下一版直接餵俾 agent。chat id 用 @userinfobot 攞最快。

# 工具 · Telegram ②(餵俾 agent · 佢自己 setup)

唔使打 /telegram:configure、唔使手機 pairing —— 將 token + chat id 俾 agent,佢自己驗 Bun(冇就裝)、寫好兩個 file。你出口,佢落手。

💡 同 AGENT 咁講(英文,連 TOKEN + CHAT ID 一齊俾)

Set up my Telegram channel. Bot token: 123456789:AAHfiqksKZ8...

My Telegram chat id: 412587349. Do EXACTLY this, in order:

1. Run `bun --version`. If Bun is missing, install it for my OS  
(Mac/Linux: `curl -fsSL https://bun.sh/install | bash;`  
Windows: `powershell -c "irm bun.sh/install.ps1 | iex"`).
  2. Write `TELEGRAM_BOT_TOKEN=<token>` to  
`~/.claude/channels/telegram/.env` (make the folder if needed).
  3. In `~/.claude/channels/telegram/access.json` set  
`"dmPolicy": "allowlist"` and add my chat id to `"allowFrom"`.
- Then tell me how to relaunch (and to restart my terminal first if you just installed Bun).

# 實作 · 整個一鍵啟動你個 agent

呢句 launch command 太新,連大 model 都未必估啱,所以直接俾佢確實命令;但點寫 .command、chmod、點用 —— 仍然 agent 落手:

💬 你同 AGENT 咁講(自然英文)

```
Make me a double-click .command launcher on my Desktop  
so I never type in the terminal. It must open Terminal  
and run EXACTLY this (and chmod itself executable):
```

```
ollama launch claude --model kimi-k3:cloud \  
-- --channels plugin:telegram@claude-plugins-official \  
--dangerously-skip-permissions
```

```
That = Ollama brain + Telegram + auto-accept. Then tell  
me how to use it.
```

確實命令直接畀(syntax 太新);其餘整檔、chmod +x、放 Desktop、點用 —— agent 落手。你出口,佢落手。

# 實作 · 喺 agent 個 folder 起好結構

我哋已經喺 agent 個 folder 入面,直接喺度起好個 workspace。⚠️ cloud model ( kimi-k3:cloud 等 ) 雖然夠叻,但仍未必似 Claude 咁識自己估,要明講 **CLAUDE.md / SOUL.md / MEMORY.md** 係咩、要做乜——唔好寫得太含糊。

💡 喺 CLAUDE CODE 入面打 ( 英文 · 清楚列明檔案 )

Set up my AI workspace in this folder. Do exactly this:

1. Create 3 sub-folders here: work, family, personal.
2. In EACH sub-folder, create CLAUDE.md (how to behave there).
3. Here in the main folder, create 3 files:  
CLAUDE.md = master guide + index to each area  
SOUL.md = your identity: name, personality, tone  
MEMORY.md = long-term memory of me; keep adding to it.
4. Ask me what goes in each file, then show me the tree.

🔑 好 prompt = 清楚 + 具體。結構同慣例 ( 檔名、位置 ) 要明示,尤其對細 model 先做得啱。

# 部機要 24/7 開住 · 唔好瞓著(記得插電)

## Windows

- › Settings → System → Power & battery → Screen and sleep: 「Sleep」設 Never(插電)
- › Lid:Power & battery → Lid & power button controls → 關蓋(插電)= Do nothing
- › 快捷:powercfg /change standby-timeout-ac 0

## Mac

- › System Settings → Battery → Options:插電時 Prevent automatic sleeping when display off
- › 關蓋都唔瞓:terminal 打 sudo pmset -a disablesleep 1
- › (還原:sudo pmset -a disablesleep 0)

# Chrome Remote Desktop · 隨時遠端睇部機

---

## 1 部機裝 host

Chrome 開 [remotedesktop.google.com/access](https://remotedesktop.google.com/access) → Set up for remote access → Download → 改名 + 設 PIN(≥6 位)

## 2 同一個 Google 戶口

部機 + 手機/另一部電腦都 login 同一個 Google account;部機要保持開住

## 3 遠端連線

另一部電腦或手機 app 開 [remotedesktop.google.com/access](https://remotedesktop.google.com/access) → 揀部機 → 入 PIN

## 4 用嚟做咩

隨時睇 agent 做緊乜、開關 session、check cron 出唔出事

# 成果 · 你今堂搞咗咩

---

-  一個喺自己機跑、用 kimi-k3:cloud ( Ollama cloud ) 嘅 **working agent**
-  一個多領域 workspace ( work / family / personal ) + **CLAUDE.md / SOUL.md / MEMORY.md**
-   Telegram ( 手機對話 ) + 一鍵啟動 launcher
-  學會核心方法：用英文 **prompt** 叫 **agent** 做,唔使自己寫檔
-  唔使 ChatGPT / Claude 戶口,香港用到,月費可 \$0



# 功課 1 · 養熟你個 agent(教佢識你)

一個禮拜內不斷同佢傾,叫佢逐點記入 MEMORY.md / SOUL.md —— 參考 LillyRose 點識得佢主人

- 你係邊個 — 名、做咩工、屋企人、講咩語言(例:「同我用廣東話」)
- 你嘅習慣同喜好 — 幾點瞓、鐘意點傾偈、口味、興趣
- 你做緊咩 — 手上嘅 project、近排關心嘅嘢(下一份功課就揀)
- 佢嘅性格 — 改個名俾佢、想佢咩語氣,寫入 **SOUL.md**
- 👉 一週後叫佢講返「你記得我啲咩」,睇佢有幾識你 —— 全程用英文 / 廣東話講,佢落手



## 功課 2 · 揀定你嘅 Project(貫穿五堂)

揀一件你真係想 agent 幫你搞掂嘅事 —— 之後五堂一路 build 大佢

- 揀一件你日日 / 周周都做嘅悶事 — 例:整理崇拜詩歌、記開支、執相、計劃旅行、追小朋友功課、睇股市
- 設計成一個 **agentic workflow** — 佢要攞咩資料?用咩工具?一步步做啲乜?點先算做完?
- 唔使理 front-end / back-end —— 淨係諗清楚「我想佢幫我做到咩」,用文字寫低個構思
- 💡 LillyRose 就係咁砌大:由一個個小 project(詩歌、開支、旅行、投資...)逐個疊成你嘅左右手
- 下堂(Class 2):你要講你個 **project** 構思,再教你 agent 第一個可重用 **Skill** + 駁 **Gmail**。

記住

# You talk. It builds.

你唔使識寫 code — 你出口,佢落手。整 2-3 個你每日用嘅就夠。